#5 … or certainly in its left subtree, if it's not empty.
#6/#7 In case we are at a level such for which the partitioning is computed on a different coordinate, then we have no way of saying were the minimum lies. We have to recurse on both branches, and find the minimum of each branch.
#8 That, however, is not enough: the minimum could be current node as well, so we have to compare these 3 values. We assume that function `min` here is an overload that takes nodes as input and handles `null` by considering it larger than any non-null node.

In that example, we attempt to remove node `B`, which only has one child, and no right branch (works similarly in the symmetric case). If we tried to simply replace `B` with its children `E`, this node would appear one level up in the tree, and likewise all its children.

So, beforehand node `E` was using `x` coordinates to partition nodes in its subtree, such that node `H` was on the right of `E` because `H`'s `x` coordinate (`2.5`) is larger than `E`'s (`2`).

After we move `E` and its subtree up, we would need to use `y` coordinates to partition nodes in `E`'s subtree. But `H`'s `y` coordinate (`-8`) is larger than `E`'s (`-5`) so node `H` doesn't belong anymore to `E`'s right branch, and the k-d tree invariants are violated.

In this case it might look like something easy to fix, but we would need to re-evaluate every single node in `E`'s subtree, and rebuild it.